



Iterative Method: Pagerank

Hoa Vu

Algorithms for Big Data

SDSU, 2025

Logistics

- Canvas quizzes (Weekly 50%)
- 2 problem sets (due at mid-semester and end of semester) (Weekly 50%)
- Main textbook: Mining of Massive Datasets (Leskovec, Rajaraman, Ullman)
<http://www.mmds.org/>
- Topics:
 - Iterative methods (pagerank, q-learning)
 - Data Stream Algorithms
 - Map-Reduce
 - Clustering
 - Recommendation algorithms
 - Gradient descent
 - Large scale machine learning

Quantifying Web Page Importance

- The internet contains billions of web pages, varying widely in quality and relevance.
- Even the most obscure search can return a lot of results.
"Used cars in New York City".
- Search engine must order the results in a way that is useful to the user.
- Why is `https://www.bloomberg.com/` is more reputable than say `https://www.mrstocktrader.com/?`

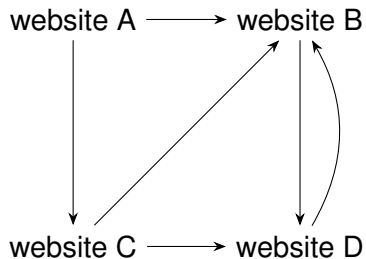
Reputation System

- Exists in many social networks from hotel reviews to search.
- Example 1: TripAdvisor allows users to rate hotels and restaurants and search for ones in a particular location.
- Example 2: Google uses a reputation system to rank web pages.

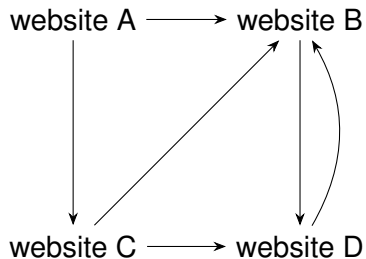
Ideas

How to rank search results?

Naive PageRank

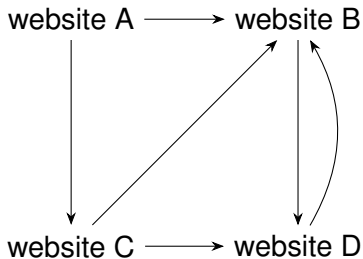


Naive PageRank

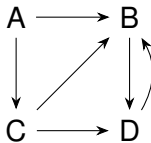


Naive PageRank

- If A links to B , then A is said to think highly of B .
- If many pages link to B , then B 's reputation is high.



Naive PageRank



$$\pi_A = 0$$

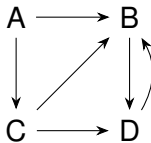
$$\pi_B = \frac{1}{2}\pi_A + \frac{1}{2}\pi_C + \pi_D$$

$$\pi_C = \frac{1}{2}\pi_A$$

$$\pi_D = \pi_B + \frac{1}{2}\pi_C$$

$$\pi_A + \pi_B + \pi_C + \pi_D = 1$$

Naive PageRank



$$\pi_A = 0$$

$$\pi_B = \frac{1}{2}\pi_A + \frac{1}{2}\pi_C + \pi_D$$

$$\pi_C = \frac{1}{2}\pi_A$$

$$\pi_D = \pi_B + \frac{1}{2}\pi_C$$

$$\pi_A + \pi_B + \pi_C + \pi_D = 1$$

Naive PageRank

Definition of naive pagerank

Definition

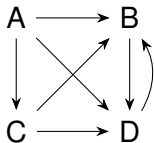
$$\pi_v = \sum_{u \rightarrow v \in E} \frac{\pi_u}{d_u}$$

Constraint: $\sum_{v \in V} \pi_v = 1$ and $\pi_v \geq 0$ for all $v \in V$.

Intuition: if u has d_u outgoing edge then u contributes $1/d_u$ to the pagerank of v .

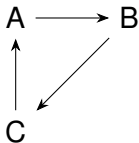
Naive PageRank

- Exercise. Compute the naive pagerank of A, B, C, D .



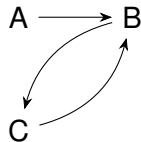
Naive PageRank's Flaws

- A webpage cannot force other webpages to link to it.
- However, it can change its pagerank by changing its outgoing links.



- In the above, the pagerank of each page is $1/3$.
- Let see what happens if *C* changes its link to *A* to *B*.

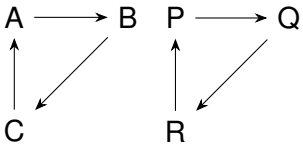
Naive PageRank's Flaws



- Since A has no link to it, its pagerank is now 0.
- The pagerank of B and C is now $1/2$.
- Thus, C can “steal” pagerank from A .

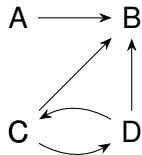
Naive PageRank's Flaws (Disconnected Components)

- An even more serious flaw is that pagerank may not be well-defined.

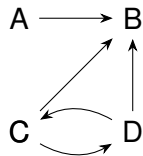


Naive PageRank's Flaws (Dangling nodes)

- An even more serious flaw is that pagerank may not be well-defined.

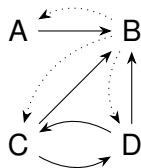


Naive PageRank's Flaws (Dangling nodes)



Fixing PageRank I

- Imagine an edge from the sink to every other node.
- In the example below, now $d_b = 3$.

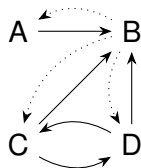


Fixing PageRank II

- Allow teleport. Each node now follows an outgoing link with α probability or teleport to a random node.

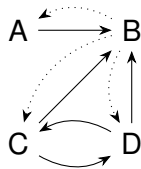
$$\pi_v = \alpha \left(\sum_{u \rightarrow v \in E} \frac{\pi_u}{d_u} \right) + \frac{1 - \alpha}{n}$$

- Google set $\alpha \approx 0.85$.



$$\pi_c = \alpha(\pi_b/4 + \pi_d/2) + (1 - \alpha)/4.$$

Fixing PageRank



$$\pi_A = 0.85 \cdot \left(\frac{1}{3} \pi_B \right) + 0.15 \cdot \frac{1}{4}$$

$$\pi_B = 0.85 \cdot \left(\pi_A + \frac{1}{2} \pi_C + \frac{1}{2} \pi_D \right) + 0.15 \cdot \frac{1}{4}$$

$$\pi_C = 0.85 \cdot \left(\frac{1}{3} \pi_B + \frac{1}{2} \pi_D \right) + 0.15 \cdot \frac{1}{4}$$

$$\pi_D = 0.85 \cdot \left(\frac{1}{3} \pi_B + \frac{1}{2} \pi_C \right) + 0.15 \cdot \frac{1}{4}$$

Solving system of linear equations with Python

$$\begin{bmatrix} \pi_A \\ \pi_B \\ \pi_C \\ \pi_D \end{bmatrix} = 0.85 \cdot \begin{bmatrix} 0 & \frac{1}{3} & 0 & 0 \\ 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & \frac{1}{3} & 0 & \frac{1}{2} \\ 0 & \frac{1}{3} & \frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} \pi_A \\ \pi_B \\ \pi_C \\ \pi_D \end{bmatrix} + 0.15 \cdot \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}$$

Solving system of linear equations with Python

$$\begin{bmatrix} \pi_A \\ \pi_B \\ \pi_C \\ \pi_D \end{bmatrix} = 0.85 \cdot \begin{bmatrix} 0 & \frac{1}{3} & 0 & 0 \\ 1 & 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & \frac{1}{3} & 0 & \frac{1}{2} \\ 0 & \frac{1}{3} & 0 & 0 \end{bmatrix} \begin{bmatrix} \pi_A \\ \pi_B \\ \pi_C \\ \pi_D \end{bmatrix} + 0.15 \cdot \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}$$

$$\pi = \alpha M \pi + (1 - \alpha)/n \cdot \mathbf{1} \iff (\alpha M - I) \pi = -(1 - \alpha)/n \cdot \mathbf{1}$$

PageRank Calculation in Python

```
import numpy as np
```

```
alpha = 0.85
```

```
n = 4
```

```
M = np.array([
    [0, 1/3, 0, 0],
    [1, 0, 1/2, 1/2],
    [0, 1/3, 0, 1/2],
    [0, 1/3, 1/2, 0]
], dtype=float)
```

```
I = np.eye(n)
```

```
lhs = I - alpha * M
```

```
rhs = (1 - alpha)/n * np.ones((n, 1))
```

0.

0.0.

PageRank Calculation in Python

```
[fontsize=\small, linenos]{python}
```

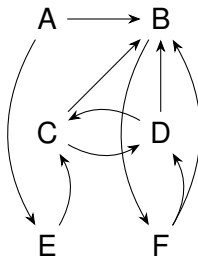
PageRank vector:

```
[[0.1414085 ]  
 [0.36673587]  
 [0.24592782]  
 [0.24592782]]
```

Problem Set

Problem 1.

Use Python, compute the pagerank of the nodes of this graph with $\alpha = 0.15$.



Problem Set

Problem 2.

Come up with a network with A, B, C, D, E such that for damping factor $\alpha = 1$, such that $\pi_A = \pi_B = \pi_C = 2/9$ and $\pi_D = 1/3$.

Problem Set

Problem 3. Suppose there is a network with n nodes, each with outgoing links to every other node.

- Without any calculation, what is the PageRank of each node?
- Now assume $\alpha = 1$. Suppose one particular node removes all its outgoing links and instead points only to itself. What is its new PageRank (express in terms of n)?
- Generalize the above for arbitrary damping factor $\alpha \in (0, 1)$.

